

Data Preprocessing in Machine Learning

From Raw Data → Model-Ready Data

 Beginner-Friendly Guide

 Practical Python Code

 Real-World Examples

 pandas + scikit-learn

Introduction to Data Preprocessing

What is it? And why do we need it?

WHAT is Data Preprocessing?

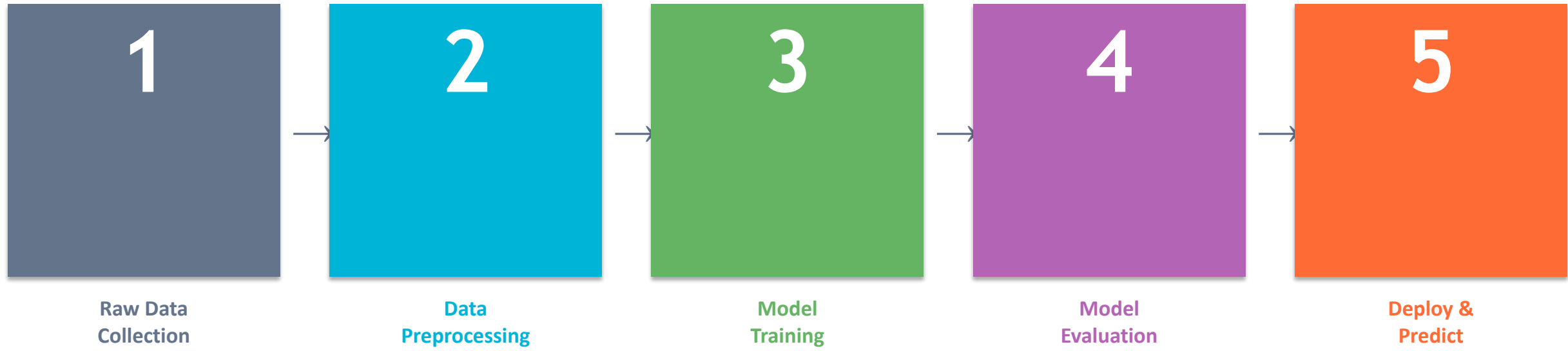
- ▶ The process of cleaning & organizing raw data
- ▶ Makes data usable for ML algorithms
- ▶ Handles errors, gaps, and inconsistencies
- ▶ Transforms data into the right format
- ▶ First & most critical step in any ML project

WHY is it Important?

- ▶ Real data is messy — missing, wrong, noisy
- ▶ Garbage In = Garbage Out (GIGO principle)
- ▶ Improves model accuracy significantly
- ▶ Reduces training time & computation cost
- ▶ 80% of a data scientist's time is spent here!

Importance in the ML Pipeline

Where does preprocessing fit?



★ *Preprocessing is highlighted — it directly impacts all downstream steps!*

✓ Better Accuracy

✓ Faster Training

✓ Less Overfitting

✓ Cleaner Insights

Types of Data in Machine Learning

Know your data before you process it!



Numerical

- ▶ Numbers — integers or decimals
- ▶ Age: 25, Salary: ₹50,000
- ▶ Temperature: 36.6°C
- ▶ Can be → Discrete or Continuous



Categorical

- ▶ Labels / Categories
- ▶ Gender: Male/Female
- ▶ City: Delhi, Mumbai
- ▶ No natural numeric order



Time Series

- ▶ Data ordered by time
- ▶ Stock price over 30 days
- ▶ Daily temperature readings
- ▶ Sequence & trends matter



Text Data

- ▶ Unstructured words/sentences
- ▶ Customer reviews
- ▶ Emails, tweets, comments
- ▶ Needs NLP techniques

Handling Missing Values

What to do when data is incomplete?

Why Does Data Go Missing?

Survey skipped • Sensor failure • Data entry error • Record not found



Example: Student Data

Student	Marks	Age
Alice	85	22
Bob	✗ NaN	25
Carol	90	✗ NaN



Drop the Row

Use when: few missing values
Drop Bob's record if < 5% missing



Fill with Mean

Use when: numerical data
Fill Marks NaN → Mean = 87.5



Fill with Mode

Use when: categorical data
Fill City NaN → Most common city



Predict the Value

Use when: many missing values
Use another ML model to predict

Handling Outliers

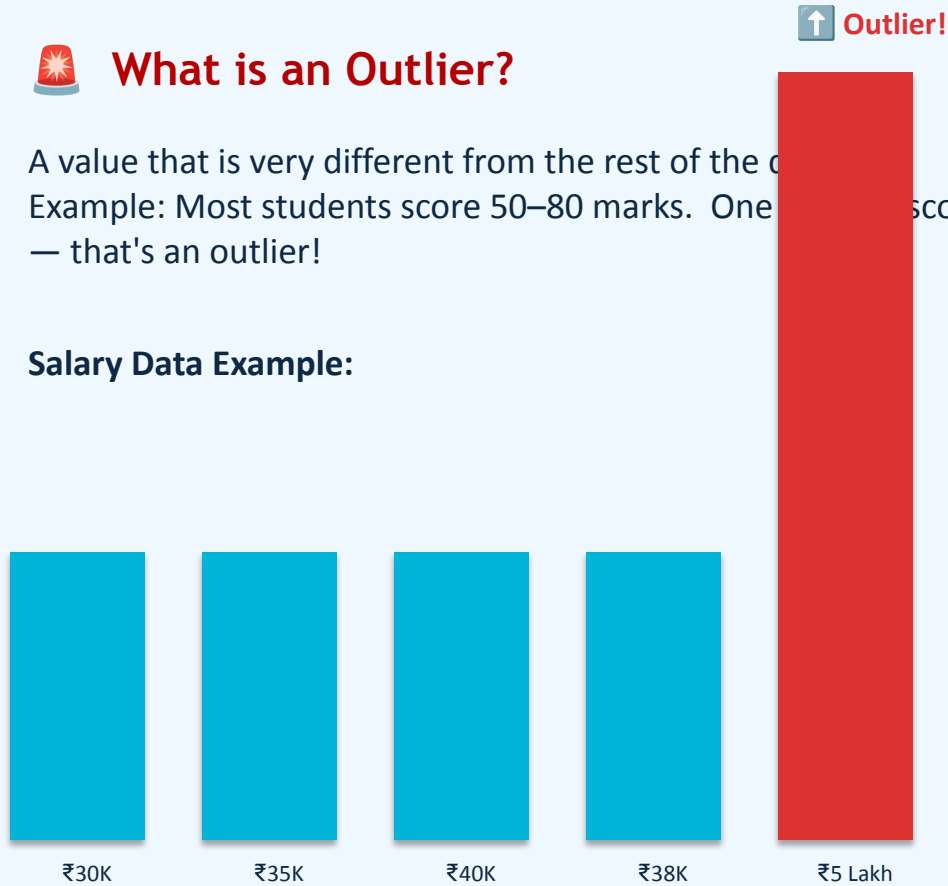
Extreme values that can fool your model!



What is an Outlier?

A value that is very different from the rest of the data.
Example: Most students score 50–80 marks. One student scores 2 marks — that's an outlier!

Salary Data Example:



IQR Method

Q1 & Q3 define normal range
Outside $1.5 \times \text{IQR}$ = outlier
Most popular approach



Z-Score Method

Measures how far from mean
 $|Z| > 3$ = outlier
Works for normal distributions



What to Do?

Remove: if clearly wrong data
Cap/Floor: limit to a max value
Keep: if real extreme events

Encoding Categorical Data

Turning text labels into numbers for ML models

🤔 Why Encode? ML models only understand numbers — not words like 'Male' or 'Delhi'!

1 Label Encoding

Assigns a number to each category

Color	→	Color Number
Red	→	Red 0
Green	→	Green 1
Blue	→	Blue 2

- ▶ ✓ Simple & fast
- ▶ ⚠ Problem: Model may think $2 > 1 > 0$
- ▶ Best for: Ordinal data (Low/Med/High)

2 One-Hot Encoding

Creates a new column for each category (0 or 1)

Color	Is_Red	Is_Green	Is_Blue
Red	1	0	0
Green	0	1	0
Blue	0	0	1

- ▶ ✓ No false ordering between categories
- ▶ ⚠ Problem: Many columns if many categories
- ▶ Best for: Nominal data (Gender, City, Color)

Feature Scaling

Making all features play fair — no feature should dominate!

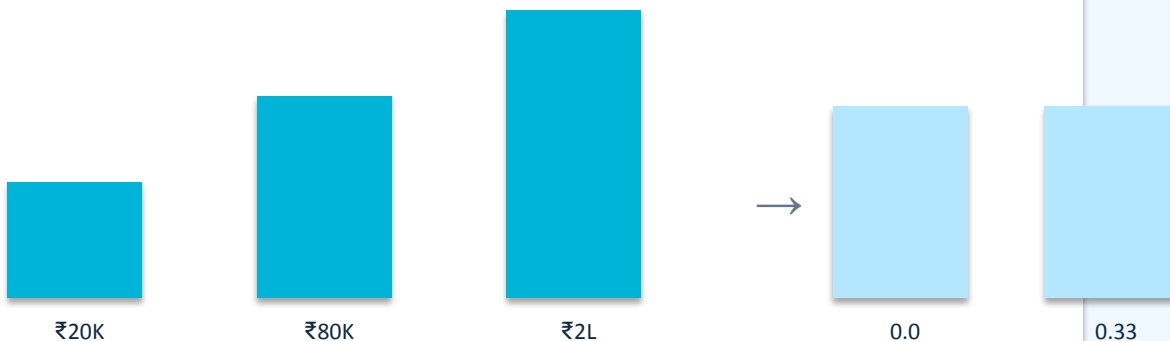
Problem: Age (20–60) vs. Salary (₹20,000 – ₹2,00,000) — Salary dominates unfairly!



Normalization (Min-Max)

Formula: $X_{\text{scaled}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$

- ▶ Scales all values to range: 0 to 1
- ▶ Example: Salary ₹20K → 0.0, ₹2L → 1.0
- ▶ Great for Neural Networks & KNN
- ▶ ⚠ Sensitive to outliers



Standardization (Z-Score)

Formula: $X_{\text{scaled}} = (X - \text{mean}) / \text{std_dev}$

- ▶ Centers data: mean=0, std=1
- ▶ Example: Marks become 0.0, -1.2, +1.5
- ▶ Great for SVM, Logistic Regression
- ▶ ✓ Not affected by outliers much



Quick Rule:

Normalization → 0 to 1 range | Standardization → mean 0, std 1

Feature Engineering

Creating smarter features to boost model performance

💡 *Feature Engineering = Using your domain knowledge to create better input features for the model*

+ Feature Creation

- ▶ Date → Day, Month, Year separately
- ▶ Name → Name length as a feature
- ▶ Price & Qty → Revenue = Price × Qty

✂ Feature Selection

- ▶ Drop irrelevant columns (e.g., ID)
- ▶ Keep only correlated features
- ▶ Reduces noise & overfitting

🔗 Feature Combination

- ▶ Age + Income → Affordability Score
- ▶ Height + Weight → BMI
- ▶ Combine related features logically

📦 Binning

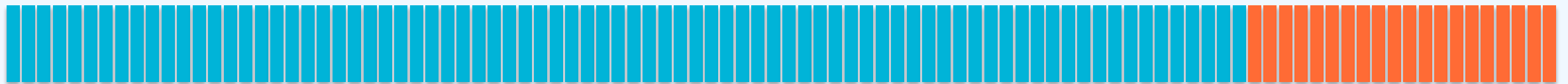
- ▶ Age: 0-18→Child, 18-60→Adult
- ▶ Marks: 90-100→Grade A, 80-89→B
- ▶ Converts continuous → categorical

Train-Test Split

Teaching and Testing your model fairly

🎯 *Why Split Data? You need separate data to train the model AND to test if it actually learned!*

Your Full Dataset (e.g., 1000 student records)



← Training Data (80%): 800 Records →

Test
(20%)

Training Set (80%)

Model **LEARNS** patterns from this data
→ Think: Studying from textbook

Test Set (20%)

Model is **EVALUATED**
on unseen data
→ Like exam day!

- ▶ Common split: 80/20 or 70/30
- ▶ Always shuffle before splitting (use `random_state` for reproducibility)
- ▶ Never train and test on the same data — that's cheating!
- ▶ For small datasets: use K-Fold Cross Validation

Data Leakage

The hidden enemy of machine learning models

 **Data Leakage = When your model accidentally 'sees' test data during training → gives fake high accuracy!**

Real Life Analogy:

A student memorizes the EXACT exam answers before the test.

They score 100% — but learned nothing!

In ML: if test data leaks into training, model looks great but fails in real world.

How Leakage Happens:

- ▶ Scaling BEFORE splitting (wrong!)
- ▶ Imputing missing values on full data
- ▶ Including future info (e.g., 'loan_default' to predict default)
- ▶ Duplicate records in train and test

How to Prevent:

- ▶ Always split data FIRST, then preprocess
- ▶ Fit scaler/encoder only on training data
- ▶ Use Pipeline in scikit-learn

Golden Rule:

Split → Fit on Train → Transform Both

NEVER fit on test data!

NEVER use full data for fit_transform!

Python — Handling Missing Values

Using pandas & scikit-learn

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer

# Sample student dataset
data = {'Name':    ['Alice', 'Bob', 'Carol', 'David'],
        'Marks':  [85, np.nan, 90, 78],
        'Age':    [22, 25, np.nan, 23]}

df = pd.DataFrame(data)
print("Before:\n", df)

# Check missing values
print("\nMissing values:\n", df.isnull().sum())

# Fill Marks with mean using SimpleImputer
imputer = SimpleImputer(strategy='mean')
df['Marks'] = imputer.fit_transform(df[['Marks']])

# Fill Age with median
df['Age'].fillna(df['Age'].median(), inplace=True)

print("\nAfter Imputation:\n", df)
```



Expected Output:

Before:

	Name	Marks	Age
0	Alice	85.0	22.0
1	Bob	NaN	25.0
2	Carol	90.0	NaN
3	David	78.0	23.0

Missing: Marks=1, Age=1

After Imputation:

	Name	Marks	Age	
0	Alice	85.0	22.0	
1	Bob	84.3	25.0	← filled
2	Carol	90.0	23.0	← filled
3	David	78.0	23.0	

Python Implementation – Encoding Categorical Data

Label Encoding & One-Hot Encoding with scikit-learn

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import OneHotEncoder

# Sample customer data
df = pd.DataFrame({
    'Customer': ['Alice', 'Bob', 'Carol'],
    'Gender':    ['Female', 'Male', 'Female'],
    'City':      ['Delhi', 'Mumbai', 'Chennai']
})

# 1. Label Encoding (for ordinal/binary data)
le = LabelEncoder()
df['Gender_Encoded'] = le.fit_transform(df['Gender'])
# Female → 0, Male → 1

# 2. One-Hot Encoding (for nominal data)
df_encoded = pd.get_dummies(df, columns=['City'])
# Creates: City_Chennai, City_Delhi, City_Mumbai

print(df_encoded.to_string())
```



Expected Output:

	Customer	Gender	Gender_ Encoded
0	Alice	Female	0
1	Bob	Male	1
2	Carol	Female	0

	City_Chennai	City_Delhi
0	0	1
1	0	0
2	1	0

	City_Mumbai
0	0
1	1
2	0



Female=0, Male=1



City expanded to 3 columns

Python Implementation – Feature Scaling

MinMaxScaler & StandardScaler from scikit-learn

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Sample data: Age & Salary
df = pd.DataFrame({
    'Age': [22, 35, 45, 28, 52],
    'Salary': [30000, 60000, 90000, 45000, 120000]
})

# IMPORTANT: Split FIRST, then scale!
X = df[['Age', 'Salary']]
X_train, X_test = train_test_split(X, test_size=0.2,
                                   random_state=42)

# Normalization (0 to 1)
norm = MinMaxScaler()
X_train_norm = norm.fit_transform(X_train) # fit on train only
X_test_norm = norm.transform(X_test)      # transform test

# Standardization (mean=0, std=1)
std = StandardScaler()
X_train_std = std.fit_transform(X_train)
X_test_std = std.transform(X_test)

print("Normalized:\n", X_train_norm)
print("Standardized:\n", X_train_std)
```



Key Points:

Normalized:

Age	Salary
0.0	0.0
0.43	0.33
0.77	0.67

Standardized:

Age	Salary
-1.22	-1.22
0.0	0.0
1.22	1.22

★ Always use:

.fit_transform()	→ train
.transform()	→ test

⊘ Never use:

.fit_transform()	on test!
------------------	----------

(That's data leakage!)

Python Implementation – Complete Preprocessing Pipeline

Pipeline

Putting it all together with sklearn Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer

# Numeric features: impute missing + scale
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])

# Categorical features: impute + encode
cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

# Combine both
preprocessor = ColumnTransformer([
    ('num', num_pipeline, ['Age', 'Salary']),
    ('cat', cat_pipeline, ['Gender', 'City'])
])

# Fit on train, transform both
X_train_ready = preprocessor.fit_transform(X_train)
X_test_ready = preprocessor.transform(X_test)
# Model-ready! Feed into any classifier now.
```

★ Why Use Pipeline?

- ✓ Prevents data leakage automatically
- ✓ Clean, reproducible code
- ✓ Apply same steps to train & test data
- ✓ Easy to deploy to production
- ✓ Handles multiple feature types together
- 📌 Professional standard in real ML projects!

🔄 Flow:

Raw Data

- ↓ Impute missing
- ↓ Encode categories
- ↓ Scale numbers
- ↓ Feed to model ✓



Summary

Everything you've learned — in one slide!



Data Preprocessing

Cleaning & organizing raw data before ML



Encoding

Label Encoding (ordinal) / One-Hot (nominal)



Train-Test Split

80/20 split — always shuffle first!



Missing Values

Drop / Fill with mean, median, or mode



Feature Scaling

Normalize (0–1) or Standardize (mean=0)



Data Leakage

Split first, fit on train only!



Outliers

Detect via IQR or Z-score, then remove/cap



Feature Engineering

Create, combine & bin features smartly



Python Tools

pandas + scikit-learn Pipeline



Mini Practice Task

Apply what you've learned!

You have been given a Customer Dataset. Complete the preprocessing steps below:

CustomerID	Age	Salary	Gender	City	Purchased
101	25	30000	Male	Delhi	Yes
102	NaN	50000	Female	Mumbai	No
103	45	NaN	Male	NaN	Yes
104	35	80000	Female	Delhi	Yes
105	22	25000	Male	Chennai	No

Task 1: Explore

`df.isnull().sum()` — Find missing values
`df.describe()` — Check data statistics

Task 2: Clean

Fill Age NaN with median
Fill Salary NaN with mean
Fill City NaN with mode

Task 3: Encode

Label Encode: Purchased (Yes/No)
One-Hot Encode: Gender, City

Task 4: Scale

StandardScaler on: Age, Salary
Fit on train, transform test



Bonus: Build a Logistic Regression model on the preprocessed data and check accuracy!